

Penjelasan Proyek: OCR Plat Nomor Kendaraan menggunakan VLM (LM Studio)

Dokumen ini menjelaskan konsep dan cara kerja proyek OCR plat nomor kendaraan Indonesia menggunakan Visual Language Model (VLM) yang dijalankan via LM Studio dan diintegrasikan dengan Python.

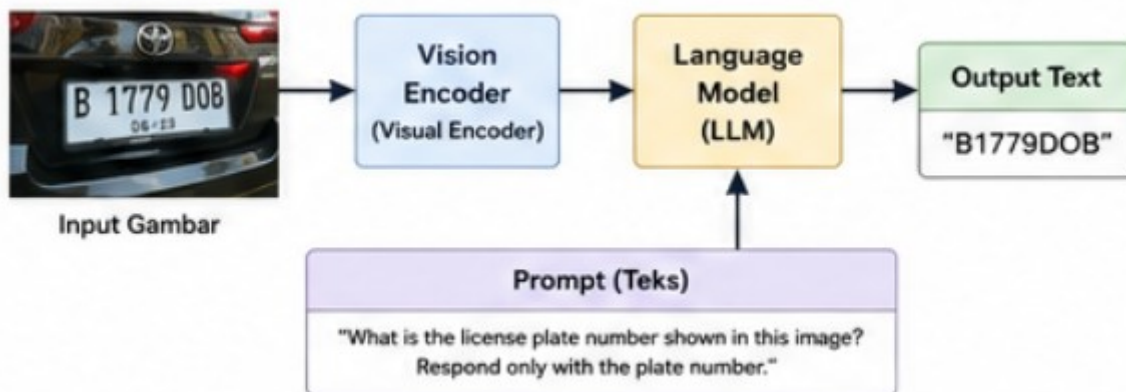
Tempat gambar ditandai dengan blok ![Deskripsi](images/nama-file.png). Simpan gambar di folder images/ (buat foldernya di root repo), lalu ganti nama-file.png sesuai nama file gambar yang kamu tambahkan.

1. Konsep VLM dan Penerapannya untuk OCR

Visual Language Model (VLM) adalah model AI yang bisa memahami gambar dan bahasa sekaligus — ia menerima input berupa gambar + teks (prompt), lalu menghasilkan output berupa teks. Ini berbeda dari model OCR tradisional (seperti Tesseract) yang murni mendeteksi dan mengenali bentuk karakter tanpa "memahami" konteks gambar secara umum.

Secara sederhana, VLM bekerja dengan:

1. **Vision encoder** — mengubah gambar menjadi representasi numerik (embedding) yang menangkap fitur visual seperti bentuk, tekstur, dan tata letak objek.
2. **Language model** — menerima embedding visual tersebut bersama token teks dari prompt, lalu memproses keduanya secara bersamaan untuk menghasilkan jawaban dalam bentuk teks.



Penerapan untuk OCR plat nomor:

VLM tidak dirancang khusus untuk OCR, tapi karena ia bisa "membaca" teks dalam gambar sebagai bagian dari pemahaman visual, ia bisa dipakai untuk tugas OCR dengan cara memberi instruksi (prompt) yang spesifik, contoh:

"What is the license plate number shown in this image? Respond only with the plate number."

Kelebihan pendekatan ini dibanding OCR konvensional:

- Tidak perlu training model khusus dari nol.
- Bisa diberi instruksi tambahan secara natural (misalnya "abaikan tanggal masa berlaku"), sesuatu yang sulit dilakukan pada OCR engine biasa.

Kekurangannya:

- Model VLM umum (general-purpose) tidak selalu seakurat model OCR khusus untuk teks pendek terstruktur seperti plat nomor.
- Perilakunya bisa dipengaruhi oleh cara prompt ditulis (lihat bagian 3).

Model yang digunakan pada proyek ini: **SmolVLM2-2.2B-Instruct**, model VLM berukuran kecil yang bisa dijalankan secara lokal tanpa GPU besar.

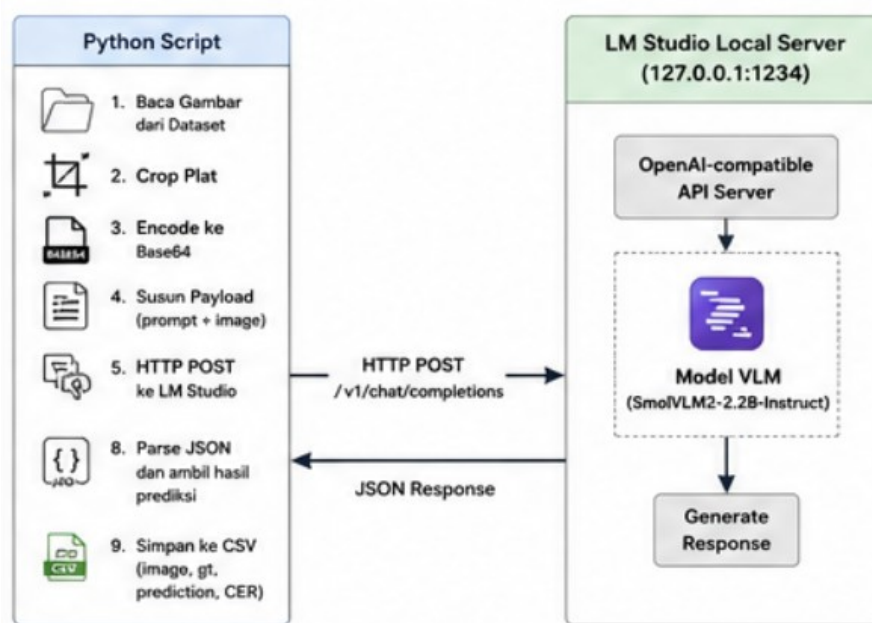
2. Cara Kerja Integrasi LM Studio dan Python

LM Studio adalah aplikasi yang memungkinkan menjalankan model AI (termasuk VLM) secara lokal di komputer, dan menyediakan **Local Server** yang meniru format API OpenAI (/v1/chat/completions). Artinya, model apa pun yang dimuat di LM Studio bisa diakses dari kode Python seolah-olah memanggil API OpenAI biasa — tanpa perlu koneksi internet atau API key.

Alur integrasinya:

[Python Script] --HTTP POST--> [LM Studio Local Server] --> [Model VLM]

[Python Script] <--JSON response-- [LM Studio Local Server] <----+



Langkah teknisnya di dalam kode:

1. **Encode gambar ke base64** — karena API mengharapkan gambar dikirim sebagai data URI (data:image/png;base64,...), bukan file biner langsung.
2. **Susun payload JSON** — berisi nama model, prompt teks, dan gambar (format image_url dalam struktur content ala OpenAI multimodal message).
3. **Kirim POST request** ke `http://127.0.0.1:1234/v1/chat/completions` menggunakan library requests.
4. **Parse response JSON** — ambil teks jawaban model dari `data["choices"][0]["message"]["content"]`.

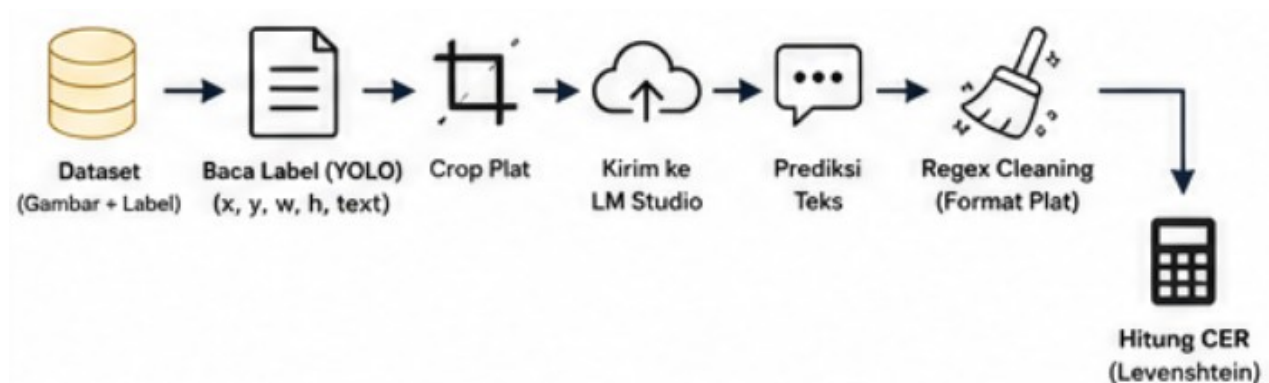
Karena formatnya mengikuti standar OpenAI, kode ini juga bisa dipakai untuk model VLM lain (misalnya Qwen2-VL, LLaVA) tanpa mengubah struktur request — cukup ganti nama model yang dimuat di LM Studio.

3. Proses Inferensi dan Evaluasi

Inferensi

Untuk setiap plat pada dataset:

1. Baca bounding box plat dari file label (labelswithLP/test/xxx.txt), yang berisi koordinat ternormalisasi (x_center, y_center, width, height) beserta teks plat asli (ground truth).
2. **Crop** gambar sesuai bounding box tersebut menjadi gambar kecil berisi plat saja — ini penting karena satu foto kendaraan bisa memuat lebih dari satu plat.
3. Kirim crop tersebut ke LM Studio dengan prompt yang meminta model membaca **hanya** nomor plat utama (mengabaikan tanggal masa berlaku).
4. Bersihkan hasil jawaban model dengan regex agar formatnya konsisten (huruf besar, tanpa simbol, tanpa teks tambahan di luar pola plat).



Evaluasi

Metrik yang digunakan adalah **Character Error Rate (CER)**:

$$\text{CER} = (S + D + I) / N$$

- **S (Substitution)** — karakter yang salah tertukar (mis. O dibaca Q)
- **D (Deletion)** — karakter yang hilang/tidak terbaca
- **I (Insertion)** — karakter tambahan yang tidak seharusnya ada

- **N** — jumlah karakter pada ground truth

CER dihitung menggunakan algoritma **edit distance (Levenshtein)** dengan proses **traceback** untuk membedakan operasi substitusi, delete, dan insert secara terpisah (bukan cuma menghitung total jarak edit).

Setiap hasil (per plat) disimpan sebagai satu baris di file CSV, dengan kolom image, ground_truth, prediction, CER_score.

4. Penjelasan Hasil CER: Contoh Sukses dan Gagal

Contoh Sukses (CER = 0.0)

image	ground_truth	prediction	CER
test008_0	DK1157AAB	DK1157AAB	0.0
test100_0	B1071PRG	B1071PRG	0.0

Pada kasus ini, prediksi **sama persis** dengan ground truth — tidak ada substitusi, penghapusan, maupun penyisipan karakter. Biasanya terjadi saat gambar plat cukup jelas (pencahayaannya baik, tidak blur, sudut kamera tidak terlalu miring) dan formatnya sesuai pola umum plat Indonesia (huruf-angka-huruf), sehingga prompt dan regex pembersih bekerja dengan baik.



Contoh Gagal Ringan (CER kecil, 0.1–0.2)

image	ground_truth	prediction	CER
test008_1	AA1997FE	AA1997EE	0.125
test100_1	B2134PZJ	82134PZJ	0.125

Di sini terjadi **1 kesalahan substitusi** karakter yang bentuknya mirip secara visual ($F \leftrightarrow E$, $B \leftrightarrow 8$). Ini konsisten dengan cara kerja VLM yang mengenali karakter dari bentuk visualnya — kesalahan semacam ini juga umum terjadi pada OCR konvensional, menandakan sumber errornya kemungkinan besar dari kualitas/resolusi gambar crop, bukan dari kegagalan model memahami instruksi.



Contoh Gagal Berat (CER tinggi, > 0.5)

image	ground_truth	prediction (sebelum perbaikan prompt)	CER
test098_0	B1779DOB	B1779DOBC628	0.5
test075_0	G2474XZ	6247X21226	1.0

Dua pola kegagalan utama yang ditemukan:

1. **Model ikut membaca tanggal masa berlaku plat** (contoh test098_0) — teks kecil di bawah nomor plat (tanggal validasi, misal "C628") ikut dianggap bagian dari jawaban. Ini terjadi karena prompt awal tidak menjelaskan bahwa ada dua jenis teks berbeda di plat. Setelah prompt diperjelas dan ditambahkan regex ekstraksi pola plat, sebagian besar kasus seperti ini berhasil diperbaiki.
2. **Model salah membaca hampir seluruh karakter** (contoh test075_0) — biasanya terjadi pada gambar dengan kualitas crop yang buruk (blur, pencahayaan sangat rendah, sudut pengambilan gambar ekstrem), sehingga model tidak bisa mengenali bentuk karakter dengan benar sama sekali. Kasus seperti ini adalah keterbatasan model VLM kecil (2.2B parameter) yang sifatnya general-purpose, bukan model OCR khusus.



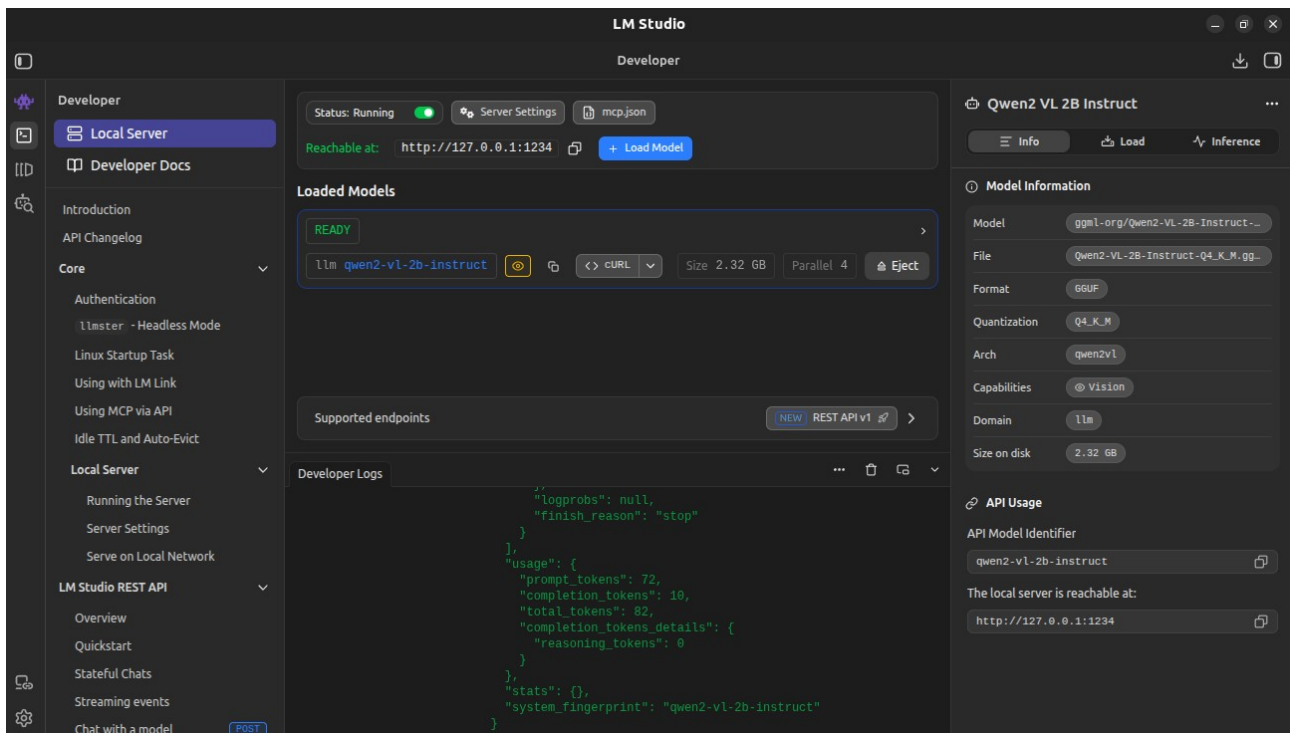
Ringkasan Distribusi Hasil

Kategori	Jumlah	Persentase
CER = 0 (sempurna)	133 / 197	67.5%
CER ≤ 0.2 (baik)	164 / 197	83.2%

Kategori	Jumlah	Persentase
CER > 0.5 (gagal berat)	17 / 197	8.6%
Rata-rata CER		0.1082

Referensi Singkat

- LM Studio Local Server (format OpenAI-compatible API)
- Konsep VLM: vision encoder + language model
- Character Error Rate (CER) untuk evaluasi OCR



Model yang digunakan di LMstudio